

Recordatorio

En las iteraciones anteriores se implementó:

- **Iteración 01:** programa en consola (CLI) dentro de un único archivo, versión que sirvió entender la mecánica básica de la ruleta.
- **Iteración 02:** ejercicio de uso de ventanas para el módulo de login y registro en Swing, con usuarios hardcoded y almacenamiento en memoria.
- **Iteración 03:** aplicación del Principio de Responsabilidad Única (SRP), separando la lógica de la ruleta en clases.
- **Iteración 04:** incorporación del patrón Modelo–Vista–Controlador (MVC) y uso de getters y setters para aplicar encapsulamiento explícitamente.

1. Objetivos

Incorporar de manera explícita las relaciones de asociación y dependencia para conseguir una mayor trazabilidad de las jugadas y un acoplamiento adecuado entre las capas del sistema.

1.1 Objetivos Específicos

1. Implementar historial de resultados por usuario

- Cada usuario debe mantener un historial con todas las jugadas realizadas.
- Agregar a la clase **Usuario** una asociación del tipo 1 a muchos con la clase **Resultado**.
- Incorporar método para recuperar el historial.
- Crear ventana para el historial, la cual pueda ser accedida por medio de un botón dentro del menú.

2. Registrar los resultados generados por la ruleta

- Cada giro de la ruleta debe producir un objeto **Resultado**.
- Establecer una asociación del tipo 1 a muchos desde **Ruleta** hacia **Resultado**, registrando automáticamente cada jugada.

3. Trazabilidad de la apuesta en cada resultado

- El objeto **Resultado** debe almacenar el **TipoApuesta** con el que se jugó.
- Esto permite vincular cada resultado con la apuesta que lo originó.

4. Dependencias entre Vista, Controlador y Modelo

- Asegurar que las ventanas dependan exclusivamente de los controladores correspondientes y no directamente del modelo.
- Asegurar que los controladores centralicen la lógica y dependan de las clases del modelo.

2. Ayudas y Sugerencias

2.1 Diagrama UML de la Aplicación

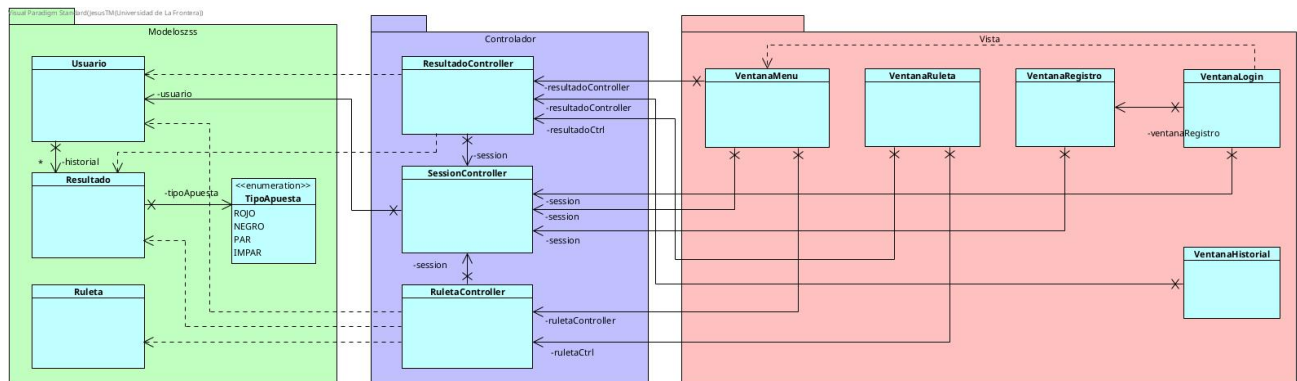


Figura 1: Diagrama de Clases - Ruleta

El diagrama anterior integra los elementos de Modelo, Controlador y Vista, destacando tanto las asociaciones como las dependencias entre clases. A nivel del paquete, **Modelo** presenta una asociación de uno a muchos entre la clase **Usuario** y la clase **Resultado**, la cual implementa el historial de jugadas. Asimismo, el ENUM **TipoApuesta** presenta los posibles valores para cada resultado.

Se debe notar que el modelo interactúa únicamente con los controladores, asegurando de esta forma que no exista acoplamiento entre la lógica de negocio y la interfaz gráfica.

En el Controlador, tanto la clases **RuletaController** y **ResultadoController** reciben al objeto **SessionController**, asegurando que las acciones del usuario se ejecuten en una misma sesión de usuario.

Los controladores actúan como intermediarios entre el modelo y la vista, manteniendo la separación de responsabilidades y mejorando la cohesión.

En la **Vista** las ventanas, `VentanaMenu`, `VentanaRuleta`, `VentanaHistorial`, dependen de los controladores y no del modelo directamente, cumpliendo con el principio de bajo acoplamiento propio de MVC.

La vista delega toda la lógica al controlador, evitando tener lógica de negocio dentro de la interfaz y limitándose únicamente a la interacción con el usuario.

Respecto a las dependencias, estas se encuentran entre las clases del paquete **Controlador** y las del paquete **Modelo**, lo cual refleja que los controladores utilizan las funcionalidades del modelo para ejecutar la lógica de negocio, sin mantener referencias a sus instancias, solo lo necesario. Por ejemplo:

- `RuletaController` depende de `Ruleta`, `Resultado` y `Usuario` para poder procesar las jugadas y registrar los resultados
- `ResultadoController` depende de `Usuario` y `Resultado` para acceder al historial.

Con esto se puede identificar que la Vista establece asociaciones con los controladores, ya que los utiliza de forma directa y persistente; en cambio, los Controladores solo dependen del modelo, delegando la lógica y manteniendo, asegurando un bajo acoplamiento.

2.2 Manejo de la sesión

Un punto que puede llegar a generar confusión corresponde al manejo de la sesión del usuario, la cual se encuentra configurada por la clase `SessionController`, la cual tiene la responsabilidad de mantener al usuario actual, autenticado, de modo que todas las operaciones que involucren historial o jugadas deben realizarse sobre la misma sesión.

En este sentido, si cada ventana de la aplicación crease una nueva instancia de `SessionController`, se producirían inconsistencias como que cada ventana trabajará como si existiera un usuario distinto, aunque sea el mismo. Para evitar este problema se debe compartir una ÚNICA instancia de `SessionController` en todo el sistema, lo cual a nivel de sistema significa:

- En los **controladores**, `RuletaController`, `ResultadoController`, se debe recibir la instancia de `SessionController` mediante el constructor:

```
public class RuletaController {  
    private final SessionController session;
```

```

    public RuletaController(Ruleta ruleta, SessionController session) {
        this.session = session; // Inyección de Dependencia
    }
}

```

- En las vistas, `VentanaRuleta`, `VentanaHistorial`, `VentanaMenu`, en lugar de crear un nuevo `SessionController`, se debe reutilizar el que fue creado al inicio de la aplicación:

```

public class VentanaMenu {
    private final SessionController session;

    public VentanaMenu(SessionController session) {
        this.session = session; // Se pasa desde el login
    }
}

```

En términos prácticos, al inicializar la aplicación se crea un único `SessionController`, el cual luego se pasa como dependencia a las demás clases:

```

SessionController session = new SessionController();
new VentanaMenu(session).mostrar();

```

De esta manera todos los controladores y ventanas trabajan con el mismo usuario.

2.3 Asociación Usuario–Resultado

La relación entre `Usuario` y `Resultado` corresponde a un ejemplo de asociación uno a muchos, al un usuario presentar múltiples resultados en su historial, mientras que cada resultado pertenece a un único usuario.

A nivel de código la implementación de la asociación se genera con el uso de una lista de resultados dentro de la clase `Usuario`.

La lista corresponde a una forma de implementar una asociación entre clases, pero se debe reconocer que existen alternativas como arreglos o colecciones.

```

public class Usuario {
    private final List<Resultado> historial = new ArrayList<>();

    public void agregarResultado(Resultado r) {

```

```

        historial.add(r);
    }

    public List<Resultado> getHistorial() {
        return Collections.unmodifiableList(historial);
    }
}

```

Finalmente, al ejecutar el siguiente fragmento de código, el historial del usuario refleja los resultados que obtiene el usuario.

```

Usuario u    = new Usuario("pepe", "123", "Pepe");
Resultado r = new Resultado(7, "ROJO", true, 1200, TipoApuesta.ROJO);
u.agregarResultado(r);
System.out.println(u.getHistorial().size());

```

2. Referencias

- [1] [Documentación] Apuntes Java - Jesús. <https://jessustm.github.io/ICC490-1-P00/>
- [2] [Documentación] Apuntes List - Jesús. <https://jessustm.github.io/ICC490-1-P00/1.Apuntes/11.List/>
- [3] [Tutorial] Vincular un proyecto Java en IntelliJ IDEA con GitHub. <https://github.com/JesusTM/ICC490-1-P00/blob/main/2.Reportes/%5BRT-P00-01%5D%20GitHub%20-%20IntelliJIDEA.pdf>
- [4] [Tutorial] Cómo usar Git y GitHub. <https://github.com/JesusTM/ICC490-1-P00/blob/main/2.Reportes/%5BRT-P00-02%5D%20Uso%20de%20GitHub.pdf>